LAPORAN RESMI
MODUL III
PERULANGAN (LOOPING)
ALGORITMA DAN DASAR PEMROGRAMAN



NAMA	:	**MAULANA MALIK ABDULLAH**
N.R.P	:	**260404100004**
DOSEN	:	**MOHAMMAD SYARIEF, ST, M.Cs**
ASISTEN	:	**MUHAMMAD YUSRI**
TGL PRAKTIKUM	:	**09 OKTOBER 2026**

Disetujui : 9 Oktober 2026
Asisten

MUHAMMAD YUSRI
230441100008



LABORATORIUM BISNIS INTELIJEN SISTEM
PRODI SISTEM INFORMASI
JURUSAN TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS TRUNODJOYO MADURA

DAFTAR ISI

DAFTAR ISI ..... **i**
BAB I PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Tujuan 1
BAB II DASAR TEORI **2**
2.1 Pengertian Perulangan 2
2.2 Perintah for 2
2.3 Perintah While 3
2.4 Perintah Break, Continue dan Pass 3
2.4.1 Perintah Break 3
2.4.2 Perintah Continue 4
2.4.3 Perintah Pass 4
BAB III TUGAS PENDAHULUAN **5**
BAB IV IMPLEMENTASI **7**
4.1 Tugas Praktikum 7
4.1.1 Tugas Praktikum Soal No. 1 7
4.1.2 Tugas Praktikum Soal No. 2 7
4.1.3 Tugas Praktikum Soal No.3 7
4.1.4 Tugas Praktikum Soal No.4 8
4.2 Source Code 9
4.2.1 Source Code Soal No.1 9
4.2.2 Source Code Soal No. 2 9
4.2.3 Source Code Soal No. 3 10
4.2.4 Source Code Soal No.4 11
4.3 Hasil 12
4.3.1 Hasil Soal No.1 12
4.3.2 Hasil Soal No.2 12
4.3.3 Hasil Soal No.3 12
4.3.4 Hasil Soal No.4 13
4.4 penjelasan 13
4.4.1 Penjelasan Soal No.1 13

4.4.2 Penjelasan Soal No. 2 14
4.4.3 Penjelasan Soal No.3 15
4.4.4 Penjelasan Soal No.4 16
BAB V PENUTUP **18**
5.1 Analisa 18
5.2 Kesimpulan 19

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman dan ilmu komputer, perulangan atau looping lahir dari kebutuhan mendasar untuk mengatasi ketidakefisienan, mencegah pengulangan kode secara manual, dan memaksimalkan potensi mesin yang memang dirancang untuk melakukan tugas repetitif tanpa rasa lelah. Konsep ini pertama kali diinisiasi oleh Ada Lovelace pada tahun 1840-an ketika ia merancang algoritma untuk mesin mekanik Analytical Engine karya Charles Babbage, di mana ia menyadari pentingnya instruksi agar mesin dapat kembali ke langkah sebelumnya untuk memproses operasi serupa. Seiring berkembangnya komputer elektronik, pada masa bahasa mesin dan Assembly, perulangan diimplementasikan secara manual menggunakan instruksi lompatan atau jump untuk mengarahkan kembali alur eksekusi program berdasarkan kondisi tertentu. Namun, penulisan kode dengan cara tersebut sangat rentan terhadap kesalahan manusia, memboroskan kapasitas memori komputer yang saat itu sangat terbatas, dan menciptakan redundansi kode yang masif jika suatu perintah harus dijalankan ribuan kali. Untuk mengatasi masalah tersebut, pada era 1950-an hingga 1960-an, para perancang bahasa pemrograman tingkat tinggi seperti Fortran dan C memperkenalkan struktur kontrol seperti for dan while untuk mewujudkan prinsip efisiensi yang dikenal sebagai Don't Repeat Yourself, sehingga manusia dapat fokus pada penyusunan logika program sementara komputer mengambil alih seluruh beban kerja repetitif secara otomatis dan presisi.

1.2 Tujuan

- Mahasiswa mampu memahami penggunaan statement perulangan dan mampu membuat program dengan konsep perulangan

BAB II
DASAR TEORI

2.1 Pengertian Perulangan

Perintah perulangan di gunakan untuk mengulang pengeksekusian statemen-statemen hingga berkali-kali sesuai dengan iterasi yang diinginkan. Dalam python, perintah untuk perulangan (loop) adalah for dan while.

2.2 Perintah for

Perintah for dalam python mempunyai ciri khas tersendiri dibandingkan dengan bahasa pemrograman lain. Tidak hanya mengulang bilangan-bilangan sebuah ekspresi aritmatik, atau memberikan keleluasaan dalam mendefinisikan iterasi perulangan dan menghentikan perulangan pada saat kondisi tertentu. Dalam python, statemen for bekerja mengulang berbagai macam tipe data sekuensial seperti List, String, dan Tuple Bentuk umum perintah for,

```
for (variabel) in (objek):
        statemen
else:
    statemen
```

Contoh 1:

```
for i in range(1,10,2):
    print(i)
```

Pada contoh 1, perintah perulangan terjadi dimana digunakan untuk menampilkan data pada pada variable I berdasrkan nilai yang ditentukan pada fungsi range() dan terdapat 3 parameter yaitu 1, sebagai nilai awal, 10 sebagai nilai akhir dan 2 sebagai nilai kenaikan.

Contoh 2:

```
for i in range (1,6):
    print("perulangan ke-", )
```

Pada contoh 2, perulangan for digunakan untuk menjalankan pengulangan dari angka 1 hingga 5 dengan kenaikan 1 sebagai default.

2.3 Perintah While

Perintah while pada python merupakan perintah yang paling umum digunakan untuk proses iterasi. Konsep sederhana dari perintah while adalah ia akan mengulang mengeksekusi statemen dalam blok while selama nilai kondisinya benar. Dan ia akan keluar atau tidak melakukan eksekusi blok statemen jika nilai kondisinya salah.Bentuk umum statemen while,

while (kondisi) :

statemen

Pada contoh pertama, merupakan contoh sederhana penggunaan while. Pada contoh di atas, program akan terus mengeksekusi statemen dalam badan while, dikarenakan kondisinya selalu benar (true). Kondisi seperti ini disebut infinite loop.

Contoh 1:

```
while x:
    print (x)
    x = x[1:]
```

Contoh 2:

```
while a < b:
    print(a)
    a = a + 1
```

2.4 Perintah Break, Continue dan Pass

2.4.1 Perintah Break

Perintah break digunakan untuk menghentikan jalannya proses iterasi pada statemen for atau while. Statemen yang berada di bawah break tidak akan di eksekusi dan program akan keluar dari proses looping.

Contoh break:

```
x = 1
while x < 5:
    if x == 3:
        break
    print(x)
    x = x + 1
else:
    print("selesai")
```

2.4.2 Perintah Continue

Statemen continue menyebabkan alur program kembali ke perintah looping. Jadi jika dalam sebuah perulangan terdapat statemen continue, maka program akan kembali ke perintah looping untuk iterasi selanjutnya.

Contoh continue:

```
n = 10
while n:
    n = n -1
    if n % 2 != 0:
        continue
    print(n)
```

2.4.3 Perintah Pass

Statemen pass mengakibatkan program tidak melakukan tindakan apa-apa. Perintah pass biasanya digunakan untuk mengabaikan suatu blok statemen perulangan, pengkondisian, class, dan fungsi yang belum didefinisikan badan programnya agar tidak terjadi error ketika proses compilasi.

Contoh program:

```
for i in range(1, 6):
    if i == 3:
        pass
    else:
        print(i)
```

Pada contoh diatas, pass digunakan sebagai perintah kosong, yaitu ketika kondisi i == 3 terpenuhi, Python tidak melakukan tindakan apa pun dan langsung melanjutkan ke perulangan berikutnya. Berbeda dengan break yang menghentikan seluruh perulangan, pass hanya melewati bagian kode yang tidak memiliki perintah.

Yauli

BAB III
TUGAS PENDAHULUAN

3.1 Soal

1. Jelaskan konsep Perulangan (for dan while) dalam bahasa Pemrograman Python menggunakan bahasamu sendiri! Apa Peran utama Perulangan dalam efisiensi penulisan kode program?
2. Buatlah sebuah Program menggunakan Perulangan for di Python untuk mencetak semua bilangan genap mulai dari angka 2 sampai 20 dan jelaskan program tersebut!
3. Apa yang dimaksud dengan kondisi infinite loop Pada Perulangan while?
   - Jelaskan faktor utama yang menyebabkannya terjadi!
   - Bagaimana cara terbaik untuk mencegah terjadinya infinite loop dalam Program Python?
4. Jelaskan fungsi dari Perulangan break, continue, dan Pass!
5. Seorang dosen ingin merekap total nilai dari beberapa tugas mahasiswa yang terkumpul. Program akan meminta input nilai tugas satu persatu. Untuk mengakhiri penginputan, dosen cukup memasukkan angka -99. Setelah selesai, Program harus menampilkan:
   - Total akumulasi nilai seluruh tugas.
   - Banyaknya tugas yang sudah diinputkan.

3.2 Jawaban

1 For untuk mengulang data yang jika jumlah pengulangannya sudah jelas. Sedangkan, while mengulang statement selama kondisinya benar, dan berhenti kalau kondisinya salah. Untuk Peran utama Perulangan yaitu kita tidak perlu menulis perintah yang sama berulang-ulang, sehingga kode lebih singkat dan rapi

2. for i in range(2, 21, 2):
    Print(i)

range disini mempunyai 3 Parameter yaitu 2 sebagai nilai awal, 21 sebagai nilai akhir, dan 2 sebagai kenaikan. Untuk Variabel i berisi angkanya, lalu Print(i) untuk mencetaknya satu Persatu

3. Infinite loop merupakan kondisi saat Perulangan while terus berjalan tanpa henti karena kondisinya selalu benar. Faktor utama penyebabnya adalah kondisi while selalu bernilai benar. Biasanya karena variabel yang di cek dalam kondisi tidak diubah didalam Perulangan atau kondisinya sengaja dibuat benar. Untuk mencegahnya yaitu dengan memastikan bahwa ada statement yang mengubah kondisi nilai variabel, bisa menggunakan break, dan bisa menekan Ctrl + C untuk menghentikannya.

4. break untuk menghentikan Proses Perulangannya, continue untuk membuat Program kembali ke awal Perulangan untuk iterasi selanjutnya, sedangkan Pass dipakai untuk blok yang isinya belum ditulis supaya tidak error, lalu program lanjut ke Perulangan berikutnya.

5. total = 0
   jumlah = 0

   nilai = int(input("Masukkan nilai tugas (-99 untuk selesai): "))

   while nilai != -99:
       total = total + nilai
       jumlah = jumlah + 1
       nilai = int(input("Masukkan nilai tugas (-99 untuk selesai): "))

   Print("Total nilai seluruh tugas:", total)
   Print("Banyaknya tugas yang diinputkan:", jumlah)

BAB IV
IMPLEMENTASI

4.1 Tugas Praktikum

4.1.1 Tugas Praktikum Soal No. 1

Pak Budi adalah kasir di Terminal Bus Sidoarjo. Tarif dihitung dari jarak tempuh penumpang:

a. Jarak ≤ 5 km: Rp3.000

b. Jarak 6 sampai 15 km: Rp3.000 + Rp1.000 untuk setiap km di atas 5

c. Jarak > 15 km: Rp13.000 + Rp500 untuk setiap km di atas 15

Setiap penumpang ke-5, ke-10, dan seterusnya mendapat diskon 50%. Kasir terus memasukkan jarak penumpang sampai memasukkan 0. Jarak negatif dianggap tidak valid dan harus diulang tanpa dihitung sebagai penumpang. Di akhir, tampilkan jumlah penumpang, total pendapatan, jarak terjauh, dan rata-rata jarak.

4.1.2 Tugas Praktikum Soal No. 2

Arsitek kota merancang Menara Lampu Hias berbentuk belah ketupat dari angka. Program meminta tinggi setengah menara n (valid jika 2 sampai 15, ulangi jika di luar rentang). Setiap baris berisi angka naik dari 1 sampai nomor baris, lalu turun kembali ke 1. Angka yang lebih dari satu digit ditampilkan sebagai angka terakhirnya saja (misalnya 10 menjadi 0). Bagian atas membesar sampai baris ke-n, kemudian mengecil kembali, dan setiap baris rata tengah.

4.1.3 Tugas Praktikum Soal No.3

Di sebuah universitas, seorang dosen ingin membuat program sederhana untuk mengelola nilai mahasiswa dalam satu kelas yang terdiri dari 10 mahasiswa. Setiap mahasiswa memiliki nilai tugas, nilai UTS, dan nilai UAS yang harus dimasukkan melalui program. Nilai akhir mahasiswa dihitung dengan bobot nilai tugas sebesar 30%, UTS sebesar 30%, dan UAS sebesar 40%. Setelah nilai akhir diperoleh, tentukan kategori nilai mahasiswa dengan ketentuan nilai 85–100 mendapatkan A, 75–84 mendapatkan B, 65–74 mendapatkan C, 50–64 mendapatkan D, dan nilai di bawah 50 mendapatkan E. Buatlah sebuah program yang dapat memproses seluruh data mahasiswa tersebut dan menampilkan nama, nilai tugas, nilai UTS, nilai UAS, nilai akhir, serta kategori nilai masing-masing mahasiswa. Setelah semua data selesai diproses, program juga harus menampilkan.

jumlah mahasiswa yang memperoleh masing-masing kategori nilai, nilai akhir tertinggi beserta nama mahasiswanya, nilaiakhir terendah beserta nama mahasiswanya, serta rata-rata nilai akhir seluruh mahasiswa

4.1.4 Tugas Praktikum Soal No.4

Sebuah universitas ingin membuat program untuk melakukan seleksi awal penerima beasiswa. Program akan memproses data 10 mahasiswa secara bergantian. Setiap mahasiswa harus memasukkan nama, IPK, dan penghasilan orang tua per bulan. Mahasiswa dengan IPK kurang dari 3,00 dianggap tidak memenuhi syarat dan datanya tidak perlu diproses lebih lanjut, sehingga program langsung melanjutkan ke mahasiswa berikutnya. Jika IPK memenuhi syarat, program akan memeriksa penghasilan orang tua. Mahasiswa dengan penghasilan orang tua kurang dari atau sama dengan Rp3.000.000 mendapatkan beasiswa penuh, sedangkan penghasilan lebih dari Rp3.000.000 sampai Rp5.000.000 mendapatkan beasiswa sebagian. Jika penghasilan lebih dari Rp5.000.000, mahasiswa tidak mendapatkan beasiswa. Program jugamemiliki batas jumlah mahasiswa yang dapat memperoleh beasiswa penuh, yaitu 3 mahasiswa. Jika sudah terdapat 3 mahasiswa yang mendapatkan beasiswa penuh, proses seleksi dapat dihentikan dan program tidak perlu memproses mahasiswa berikutnya. Buatlah program Python yang dapat menampilkan hasil seleksi setiap mahasiswa serta menggunakan mekanisme untuk melewati mahasiswa yang tidak memenuhi syarat dan menghentikan proses ketika kuota beasiswa penuh.

4.2 Source Code

4.2.1 Source Code Soal No.1

```
jumlah_penumpang = 0
total = 0
total_jarak = 0
terjauh = 0

while True:
    jarak = float(input("Masukkan jarak penumpang (km), dan 0 untuk selesai: "))

    if jarak == 0:
        break
    if jarak < 0:
        print("Jarak tidak valid, ulangi!")
        continue

    jumlah_penumpang += 1

    if jarak <= 5:
        tarif = 3000
    elif jarak <= 15:
        tarif = 3000 + 1000 * (jarak - 5)
    else:
        tarif = 13000 + 500 * (jarak - 15)

    if jumlah_penumpang % 5 == 0:
        tarif *= 0.5

    total += tarif
    total_jarak += jarak
    terjauh = max(terjauh, jarak)

rata = total_jarak / jumlah_penumpang if jumlah_penumpang > 0 else 0

print("Jumlah penumpang :", jumlah_penumpang)
print("Total pendapatan : Rp", total)
print("Jarak terjauh    :", terjauh, "km")
print("Rata-rata jarak  :", round(rata, 1), "km")
```

4.2.2 Source Code Soal No. 2

```
while True:
    n = int(input("Masukkan tinggi setengah menara (2-15): "))
    if 2 <= n <= 15:
        break
    print("Input tidak valid, ulangi!")

baris = list(range(1, n + 1)) + list(range(n - 1, 0, -1))

for i in baris:
    spasi = " " * (n - i)
    naik  = "".join(str(j % 10) for j in range(1, i + 1))   # 1 sampai i
    turun = "".join(str(j % 10) for j in range(i - 1, 0, -1))  # i-1 sampai 1
    print(spasi + naik + turun)
```

4.2.3 Source Code Soal No. 3

```
nama_list, akhir_list, kategori_list = [], [], []
jumlah = {"A": 0, "B": 0, "C": 0, "D": 0, "E": 0}

for (variable) nama: str
    nama  = input("Nama  : ")
    tugas = float(input("Tugas : "))
    uts   = float(input("UTS   : "))
    uas   = float(input("UAS   : "))

    akhir = 0.3 * tugas + 0.3 * uts + 0.4 * uas

    if akhir >= 85:
        kat = "A"
    elif akhir >= 75:
        kat = "B"
    elif akhir >= 65:
        kat = "C"
    elif akhir >= 50:
        kat = "D"
    else:
        kat = "E"

    jumlah[kat] += 1
    nama_list.append(nama)
    akhir_list.append(akhir)
    kategori_list.append(kat)

    print(f"{nama} | Tugas {tugas} | UTS {uts} | UAS {uas} | Akhir {akhir:.2f} | {kat}\n")

print("Jumlah per kategori:", jumlah)

tinggi = max(akhir_list)
rendah = min(akhir_list)
print("Tertinggi:", round(tinggi, 1), "-", nama_list[akhir_list.index(tinggi)])
print("Terendah :", round(rendah, 1), "-", nama_list[akhir_list.index(rendah)])
print("Rata-rata:", sum(akhir_list) / 10)
```

4.2.4 Source Code Soal No.4

```python
JUMLAH_MAHASISWA = 10
KUOTA_PENUH = 3
beasiswa_penuh = 0

for i in range(1, JUMLAH_MAHASISWA + 1):
    print(f"\n=== Mahasiswa ke-{i} ===")
    nama = input("Nama                        : ")
    ipk = float(input("IPK                         : "))
    penghasilan = int(input("Penghasilan orang tua/bln : Rp"))

    # Syarat IPK: jika tidak memenuhi, lewati mahasiswa ini
    if ipk < 3.00:
        print(f"{nama}: tidak memenuhi syarat (IPK < 3,00)")
        continue

    # Seleksi berdasarkan penghasilan orang tua
    if penghasilan <= 3000000:
        print(f"{nama}: mendapat BEASISWA PENUH")
        beasiswa_penuh += 1
    elif penghasilan <= 5000000:
        print(f"{nama}: mendapat BEASISWA SEBAGIAN")
    else:
        print(f"{nama}: tidak mendapat beasiswa (penghasilan > Rp5.000.000)")

    # Hentikan proses jika kuota beasiswa penuh sudah terpenuhi
    if beasiswa_penuh == KUOTA_PENUH:
        print("\nKuota beasiswa penuh (3 mahasiswa) telah terpenuhi.")
        print("Proses seleksi dihentikan.")
        break

print(f"\nTotal penerima beasiswa penuh: {beasiswa_penuh} mahasiswa")
```

4.3 Hasil

4.3.1 Hasil Soal No.1

```
Masukkan jarak penumpang (km), dan 0 untuk selesai: 6
Masukkan jarak penumpang (km), dan 0 untuk selesai: 7
Masukkan jarak penumpang (km), dan 0 untuk selesai: 5
Masukkan jarak penumpang (km), dan 0 untuk selesai: 8
Masukkan jarak penumpang (km), dan 0 untuk selesai: 0
Jumlah penumpang : 4
Total pendapatan : Rp 18000.0
Jarak terjauh    : 8.0 km
Rata-rata jarak  : 6.5 km
PS C:\Users\DANIL\AppData\Local\Programs\Microsoft VS Code>
```

4.3.2 Hasil Soal No.2

```
Masukkan tinggi setengah menara (2-15): 6
     1
    121
   12321
  1234321
 123454321
12345654321
 123454321
  1234321
   12321
    121
     1
PS C:\Users\DANIL\AppData\Local\Programs\Microsoft VS Code>
```

4.3.3 Hasil Soal No.3

```
 Nama  : zurran
 Tugas : 67
 UTS   : 87
 UAS   : 79
 zurran | Tugas 67.0 | UTS 87.0 | UAS 79.0 | Akhir 77.80 | B

 Nama  : dafa
 Tugas : 87
 UTS   : 97
 UAS   : 86
 dafa | Tugas 87.0 | UTS 97.0 | UAS 86.0 | Akhir 89.60 | A

 Jumlah per kategori: {'A': 5, 'B': 5, 'C': 0, 'D': 0, 'E': 0}
 Tertinggi: 91.1 - malik
 Terendah : 77.8 - zurran
 Rata-rata: 84.96000000000001
 PS C:\Users\DANIL\AppData\Local\Programs\Microsoft VS Code>
```

4.3.4 Hasil Soal No.4

```
=== Mahasiswa ke-1 ===
Nama                        : malik
IPK                         : 3.85
Penghasilan orang tua/bln : Rp2000000
malik: mendapat BEASISWA PENUH

=== Mahasiswa ke-2 ===
Nama                        : haikal
IPK                         : 3.56
Penghasilan orang tua/bln : Rp3500000
haikal: mendapat BEASISWA SEBAGIAN

=== Mahasiswa ke-3 ===
Nama                        : mahar
IPK                         : 3.75
Penghasilan orang tua/bln : Rp1800000
mahar: mendapat BEASISWA PENUH

=== Mahasiswa ke-4 ===
Nama                        : didin
IPK                         : 3.50
Penghasilan orang tua/bln : Rp2250000
didin: mendapat BEASISWA PENUH

Kuota beasiswa penuh (3 mahasiswa) telah terpenuhi.
Proses seleksi dihentikan.

Total penerima beasiswa penuh: 3 mahasiswa
PS C:\Users\DANIL\AppData\Local\Programs\Microsoft VS Code>
```

4.4 penjelasan

4.4.1 Penjelasan Soal No.1

Pada soal nomor 1, saya diminta membuat program untuk Pak Budi, kasir di Terminal Bus Sidoarjo, yang menghitung tarif penumpang berdasarkan jarak tempuh. Awalnya saya bingung karena jumlah penumpangnya tidak diketahui, jadi saya memakai perulangan `while True` supaya program terus meminta jarak sampai kasir mengetik 0. Berikut langkah-langkah yang saya lakukan:

- Membuat 4 variabel awal, yaitu `jumlah_penumpang`, `total`, `total_jarak`, dan `terjauh`, yang semuanya saya isi 0 dulu. Ibaratnya buku catatan kosong yang nanti diisi satu per satu setiap ada penumpang.

- Memakai `while True` untuk perulangan utama. Di dalamnya jarak diminta lewat `input()` dan diubah dengan `float()` supaya angkanya boleh desimal.
- Memakai `break`. Jika jarak sama dengan 0, berarti kasir sudah selesai, jadi perulangan dihentikan.
- Memakai `continue`. Jika jarak kurang dari 0 (tidak valid), program menampilkan tulisan "Jarak tidak valid, ulangi!" lalu langsung kembali ke awal perulangan tanpa menambah jumlah penumpang, sesuai permintaan soal.
- Menghitung tarif dengan `if - elif - else`. Jarak sampai 5 km tarifnya Rp3.000. Jarak sampai 15 km tarifnya `3000 + 1000 * (jarak - 5)`. Jarak di atas 15 km tarifnya `13000 + 500 * (jarak - 15)`.
- Memberi diskon dengan `if jumlah_penumpang % 5 == 0`. Kalau sisa baginya 0, berarti itu penumpang ke-5, ke-10, dan seterusnya, sehingga tarifnya dikali 0,5 (diskon 50%).
- Mencatat data setiap penumpang: tarif ditambahkan ke `total`, jarak ditambahkan ke `total_jarak`, dan jarak terjauh dicari dengan `max(terjauh, jarak)`.
- Setelah perulangan selesai, rata-rata jarak dihitung dengan `total_jarak / jumlah_penumpang`. Saya menambahkan `if jumlah_penumpang > 0 else 0` supaya program tidak error karena pembagian dengan nol kalau kasir langsung mengetik 0.

Pada percobaan, saya memasukkan jarak 6, 7, 5, dan 8 lalu 0. Hasilnya 4 penumpang, total pendapatan Rp18.000, jarak terjauh 8 km, dan rata-rata jarak 6,5 km. Saya coba hitung sendiri: 4.000 + 5.000 + 3.000 + 6.000 = 18.000, ternyata hasilnya sama. Karena penumpangnya baru 4, diskon penumpang ke-5 belum kepakai di percobaan ini.

4.4.2 Penjelasan Soal No. 2

Pada soal nomor 2, saya diminta membuat Menara Lampu Hias berbentuk belah ketupat dari angka. Soal ini menurut saya paling seru sekaligus paling bikin pusing karena harus memikirkan spasi dan urutan angkanya. Berikut langkah-langkahnya:

- Meminta tinggi menara dengan `while True`. Nilai `n` dimasukkan lewat `input()`, lalu dicek memakai `if 2 <= n <= 15`. Kalau benar, `break` dipakai untuk keluar. Kalau salah, program menampilkan "Input tidak valid, ulangi!" dan meminta lagi.
- Membuat daftar nomor baris: `list(range(1, n + 1)) + list(range(n - 1, 0, -1))`. Untuk n = 6 hasilnya 1, 2, 3, 4, 5, 6, 5, 4, 3, 2, 1. Jadi menara membesar dulu sampai baris ke-6, lalu mengecil lagi.
- Memakai perulangan `for i in baris`, artinya program mengerjakan satu baris untuk setiap nomor di daftar tadi.
- Membuat spasi di depan dengan `" " * (n - i)`. Makin besar nomor barisnya, makin sedikit spasinya, sehingga setiap baris terlihat rata tengah.
- Membuat angka naik dengan `range(1, i + 1)`, yaitu dari 1 sampai nomor baris. Lalu angka turun dengan `range(i - 1, 0, -1)`, yaitu dari nomor baris dikurangi 1 sampai 1.
- Memakai `j % 10` supaya angka dua digit hanya diambil angka terakhirnya (misalnya 10 menjadi 0), sesuai aturan soal. Angka-angka itu digabung menjadi satu tulisan dengan `"".join()`.
- Terakhir, spasi, angka naik, dan angka turun digabung lalu dicetak dengan `print()`.

Pada percobaan saya memasukkan n = 6, dan hasilnya menara yang berbentuk belah ketupat dengan 11 baris. Bagian atasnya 1, 121, 12321 dan seterusnya sampai 12345654321 di tengah, lalu mengecil lagi sampai 1 di bawah. Bentuknya sama seperti yang diminta soal.

4.4.3 Penjelasan Soal No.3

Pada soal nomor 3, saya diminta membuat program pengolah nilai untuk 10 mahasiswa. Karena jumlah mahasiswanya sudah pasti 10, saya memakai perulangan `for` yang berjalan 10 kali. Berikut langkah-langkahnya:

- Menyiapkan tempat penyimpanan sebelum perulangan: tiga list kosong (`nama_list`, `akhir_list`, `kategori_list`) untuk menyimpan data, dan satu dictionary `jumlah` untuk menghitung banyaknya nilai A sampai E yang awalnya semuanya 0.

- Di dalam perulangan, program meminta `nama`, `tugas`, `uts`, dan `uas`. Nilainya diubah dengan `float()` supaya bisa menerima angka desimal.
- Menghitung nilai akhir dengan rumus `0.3 * tugas + 0.3 * uts + 0.4 * uas`, sesuai bobot 30%, 30%, dan 40%.
- Menentukan kategori dengan `if - elif - else`. Nilai 85 ke atas A, 75 ke atas B, 65 ke atas C, 50 ke atas D, dan sisanya E. Urutannya dari nilai tertinggi supaya tidak salah kategori.
- Menambah hitungan kategori dengan `jumlah[kat] += 1`, lalu menyimpan nama, nilai akhir, dan kategori ke dalam list memakai `append()`.
- Mencetak data tiap mahasiswa dengan f-string. Tulisan `{akhir:.2f}` dipakai supaya nilai akhir hanya tampil 2 angka di belakang koma.
- Setelah perulangan selesai, program mencetak jumlah per kategori. Nilai tertinggi dicari dengan `max()` dan terendah dengan `min()`. Nama pemiliknya ditemukan dengan `index()`, yaitu mencari urutan nilai itu di `akhir_list` lalu mengambil nama di urutan yang sama dari `nama_list`.
- Rata-rata dihitung dengan `sum(akhir_list) / 10`, yaitu jumlah semua nilai akhir dibagi 10 mahasiswa.

Pada percobaan, contohnya zurran dengan tugas 67, UTS 87, dan UAS 79 mendapat nilai akhir 77,80 (kategori B), dan dafa dengan tugas 87, UTS 97, dan UAS 86 mendapat nilai akhir 89,60 (kategori A). Saya hitung manual hasilnya sama. Setelah 10 mahasiswa selesai, hasilnya lima orang mendapat A dan lima orang mendapat B. Nilai tertinggi 91,1 milik malik, nilai terendah 77,8 milik zurran, dan rata-ratanya 84,96. Di layar rata-ratanya tertulis 84.96000000000001. Setelah saya cari tahu, itu terjadi karena komputer menyimpan angka desimal dengan cara yang tidak selalu pas, jadi ada selisih yang sangat kecil.

4.4.4 Penjelasan Soal No.4

Pada soal nomor 4, saya diminta membuat program seleksi awal beasiswa untuk 10 mahasiswa. Di soal ini saya memakai `for`, `continue`, dan `break` sekaligus. Berikut langkah-langkahnya:

- Membuat tiga variabel di awal: `JUMLAH_MAHASISWA = 10`, `KUOTA_PENUH = 3`, dan `beasiswa_penuh = 0`. Variabel `beasiswa_penuh` dipakai untuk menghitung sudah berapa orang yang mendapat beasiswa penuh.

- Memakai `for i in range(1, JUMLAH_MAHASISWA + 1)` supaya program memproses mahasiswa ke-1 sampai ke-10. Di setiap putaran, program meminta nama, IPK, dan penghasilan orang tua.
- Memeriksa IPK dengan `if ipk < 3.00`. Kalau IPK-nya kurang dari 3,00, program menampilkan bahwa mahasiswa tersebut tidak memenuhi syarat, lalu memakai `continue` untuk langsung pindah ke mahasiswa berikutnya tanpa memeriksa penghasilan.
- Memeriksa penghasilan orang tua dengan `if - elif - else`. Penghasilan sampai Rp3.000.000 mendapat beasiswa penuh (dan `beasiswa_penuh` ditambah 1), sampai Rp5.000.000 mendapat beasiswa sebagian, dan di atas itu tidak mendapat beasiswa.
- Memeriksa kuota dengan `if beasiswa_penuh == KUOTA_PENUH`. Kalau sudah ada 3 penerima beasiswa penuh, program menampilkan bahwa kuota sudah terpenuhi lalu memakai `break` untuk menghentikan seluruh perulangan.
- Setelah perulangan selesai, program mencetak total penerima beasiswa penuh.

Pada percobaan, malik (IPK 3,85, penghasilan Rp2.000.000) mendapat beasiswa penuh, haikal (IPK 3,56, penghasilan Rp3.500.000) mendapat beasiswa sebagian, mahar (IPK 3,75, penghasilan Rp1.800.000) mendapat beasiswa penuh, dan didin (IPK 3,50, penghasilan Rp2.250.000) mendapat beasiswa penuh. Setelah didin, kuota beasiswa penuh sudah 3 orang, sehingga proses berhenti di mahasiswa ke-4 dan mahasiswa ke-5 sampai ke-10 tidak diproses lagi. Dari sini saya paham bedanya `continue` yang hanya melewati satu mahasiswa dengan `break` yang menghentikan semuanya.

BAB V
PENUTUP

5.1 Analisa

Setelah mengerjakan empat soal praktikum ini, saya jadi tahu bahwa perulangan (*looping*) itu sangat membantu. Kalau tidak ada perulangan, untuk memproses 10 mahasiswa saya harus menulis kode yang sama sebanyak 10 kali. Dengan `for` cukup ditulis sekali saja, dan komputer yang mengulanginya.

Dari keempat soal, saya belajar kapan harus memakai `for` dan kapan memakai `while`. Pada soal 3 dan 4, jumlah mahasiswanya sudah pasti 10, jadi saya memakai `for`. Pada soal 1, jumlah penumpang tidak diketahui karena kasir berhenti kalau mengetik 0, jadi saya memakai `while True`. Pada soal 2 dan soal 1, `while True` juga dipakai untuk memeriksa input supaya diulang terus sampai nilainya benar.

Saya juga belajar tentang `break` dan `continue`. Awalnya keduanya terlihat mirip, tetapi ternyata berbeda. `break` menghentikan seluruh perulangan, contohnya saat kasir mengetik 0 di soal 1 dan saat kuota beasiswa penuh sudah terpenuhi di soal 4. Sedangkan `continue` hanya melewati putaran yang sedang berjalan, contohnya saat jarak negatif di soal 1 dan saat IPK kurang dari 3,00 di soal 4.

Selain itu, perulangan ternyata sering dipakai bersama hal lain yang sudah dipelajari sebelumnya, seperti `if - elif - else`, operator `%` (untuk mencari kelipatan 5 dan mengambil angka terakhir), list, dictionary, serta fungsi `max()`, `min()`, dan `sum()`. Kalau digabung, program yang kelihatannya rumit jadi bisa dikerjakan langkah demi langkah.

Ada juga beberapa hal yang saya temukan saat mencoba program. Pertama, di soal 3 rata-rata tampil sebagai 84.96000000000001, padahal seharusnya 84,96. Kemungkinan ini bisa dirapikan dengan fungsi `round()`. Kedua, program saya belum bisa menangani kalau pengguna salah mengetik huruf saat diminta angka, karena `int()` dan `float()` akan error. Kalau belajar lebih lanjut, mungkin hal ini bisa diperbaiki supaya programnya lebih aman. Ketiga, `while True` harus selalu punya `break`, kalau tidak program akan berjalan terus tanpa berhenti (*infinite loop*).

5.2 Kesimpulan

Dari praktikum Modul III ini, saya dapat menyimpulkan bahwa perulangan (*looping*) adalah cara agar komputer mengerjakan perintah yang sama berulang-ulang secara otomatis, sehingga kita tidak perlu menulis kode yang sama berkali-kali.

Poin-poin yang bisa saya simpulkan:

- `for` dipakai ketika jumlah pengulangannya sudah jelas, seperti memproses 10 mahasiswa pada soal 3 dan 4 dan membuat baris menara pada soal 2.
- `while` dipakai ketika jumlah pengulangannya belum pasti, seperti menunggu kasir mengetik 0 pada soal 1 dan memeriksa input yang harus diulang sampai benar.
- `break` dipakai untuk menghentikan seluruh perulangan, sedangkan `continue` dipakai untuk melewati satu putaran saja lalu lanjut ke putaran berikutnya.
- Perulangan bisa digabung dengan `if - elif - else`, operator `%`, list, dictionary, dan fungsi `max()`, `min()`, serta `sum()` untuk menyelesaikan soal yang lebih rumit.
- Perulangan `while True` harus punya `break` supaya tidak terjadi *infinite loop*, dan input dari pengguna sebaiknya diperiksa supaya program tidak mudah error.